

Detailed Summary of the Article

Introduction

Convolutional Neural Networks (CNNs) have significantly advanced artificial intelligence applications, including audio, text, and visual object recognition, achieving remarkable accuracy. This success is attributed to the CNN's ability to recognize features in input data through extensive arithmetic operations involving millions to billions of trained feature parameters. However, this high accuracy comes at the cost of substantial computational complexity and massive data movements, leading to performance degradation and increased energy consumption.

Challenges in CNN Implementation

The high computational demands and energy consumption of CNNs make them unsuitable for efficient hardware implementation in battery-operated devices widely used in modern edge applications such as the Internet of Things (IoT), autonomous electric vehicles, and implanted biomedical devices. These applications require high computation throughput and low power consumption, making energy efficiency and substantial throughput critical requirements for CNN hardware accelerators.

Existing Solutions and Their Limitations

1. ***GPU and TPU***: While GPUs and TPUs provide significant computational power, they consume hundreds of watts, rendering them impractical for edge applications.
2. ***ASIC and iMC***: These offer high energy efficiency but suffer from poor compatibility and scalability with rapidly evolving CNN models due to their lack of reprogrammability.
3. ***FPGAs***: They provide a balance by offering reprogrammability and, with well-designed hardware architecture, can deliver adequate energy efficiency. Modern FPGA boards, such as the Zynq-7000 SoC and Zynq Ultrascale+ MPSoC series, can function as complete stand-alone systems for various applications.

Enhancing Energy Efficiency

Energy efficiency in high-throughput parallel-computing CNN accelerators can be enhanced by:

- Maximizing local data reuse.
- Minimizing expensive read/write operations to and from external memory.

- Utilizing low precision computation.

Prior Research

1. **Row-Stationary Data Flow**: Achieves energy efficiency by reducing off-chip memory access but is not ideal for FPGA applications.
2. **Lower Precision Computation**: Improves energy efficiency but often at the cost of accuracy, which can be mitigated with mixed precision and layer-specific optimization.
3. **Software-Based Reconfigurability**: Uses dual-loop arithmetic modules but at the expense of throughput and energy efficiency due to unoptimized data movement.
4. **Kernel Partitioning Technique**: Enhances energy efficiency by reducing off-chip memory access but requires extra preprocessing and postprocessing steps.

Key Issues

Despite advancements in parallel computing, real-world computational throughput of most CNN accelerators often falls below theoretical estimates due to underutilization of available resources and interrupted data supply. For instance, a study found that a complex model like GoogleNet utilizes only 6.6% of processing elements (PEs) in an accelerator in worst-case scenarios.

Contributions of the Article

To address these challenges, the article presents an energy-efficient, high-throughput, and reconfigurable FPGA-based CNN accelerator for object recognition applications. The main contributions are:

1. **Uninterrupted Processing Technique**: Reduces latency, enhancing throughput and energy efficiency.
2. **Data-Reusability Process**: Maximizes reuse of local data without data shifting, reducing data movement and energy consumption.
3. **Random Access Line-Buffer (RALB)-Based Hardware Architecture**: Supports the proposed techniques.
4. **Hardware Implementation and Comparison**: Implemented on various FPGA platforms and compared with state-of-the-art implementations.
5. **Functional Validation**: The hardware prototype on the Zynq-UltraScale+ MPSoC-ZCU102 FPGA board is validated in a real-world setup, classifying objects using the GoogLeNet CNN model.

Organization of the Article

- *Section II*: Provides a system-level view of the work, including mathematical background and research challenges.
- *Section III*: Discusses the proposed technique and new VLSI architectures for the CNN accelerator.
- *Section IV*: Presents implementation results, discussion, and comparison with other implementations.
- *Section V*: Concludes the article.

Conclusion

The proposed FPGA-based CNN accelerator addresses critical issues of energy efficiency and throughput by introducing innovative techniques for uninterrupted processing and data reusability. The random access line-buffer hardware architecture further enhances these benefits. The practical

Detailed Summary of Section II: System Model, Mathematical Prerequisite, and Research Challenges

A. System Model

The hardware system for an object detection application is illustrated, involving four main components: a camera, an FPGA board (Zynq MPSoC evaluation board), external memory (SD card), and a display. The FPGA board, featuring an onboard computer, integrates an ARM processor with six cores, DDR4 RAM, and various peripheral controllers (USB, SD-card, and display controllers).

- *FPGA Board Configuration*: The board contains configurable logic blocks, multipliers, and embedded block RAMs for the CNN accelerator. The SD card holds pretrained model parameters and software programs.
- *Operation Overview*: The CNN accelerator, designed using a Local Memory and Control Unit (LMCU) and an array of Processing Elements (PEs), operates iteratively. Each PE includes a Multiply-and-Accumulate (MAC) unit and a comparator unit. The LMCU manages the PE array operations and moderates data flow.
- *Data Flow*: The PE array receives input feature matrix (i -), filter weights (w), and biases (b) from the LMCU. It then produces the output feature matrix (o -), which is reused as i - for

subsequent layers. This iterative process continues until all layer operations are completed. The CNN accelerator interfaces with the onboard computer via AXI buses.

- ***Initialization and Processing***: The system initializes by transferring model information and trained parameters to configure the CNN accelerator. Once configured, the onboard computer processes input images from the camera, feeding them to the CNN accelerator. The accelerator performs operations on the images across convolutional (CONV) and fully connected (FC) layers to classify objects and compute their probabilities.

B. Mathematical Prerequisite for CNN-Accelerator Computations

The CNN accelerator performs various operations for CONV layers, including MAC, pooling, and activation operations:

1. ***MAC Operation***: Conducted using a strided sliding window-based approach to search for specific features, mathematically expressed as:

$$\begin{aligned} & \backslash[\\ & C_{\{o\}}^{x,y,z} = \sum_{p=1}^X \sum_{q=1}^X \sum_{r=1}^D (W_{\{p,q,r\}} \times i_{\{a,b,r\}}) + b_z \\ & \backslash] \end{aligned}$$

where $(C_{\{o\}})$ represents the 3-D matrix of detection scores for features searched by filters in the (l) -th CONV layer, and $(a = s \times x + p)$, $(b = s \times y + q)$.

2. ***Pooling Operation***: Max pooling maximizes prominent scores in $(C_{\{o\}})$, expressed as:

$$\begin{aligned} & \backslash[\\ & P_{\{o\}}^{x,y,z} = \max(C_{\{o\}}(x : \alpha, y : \beta, z)) \\ & \backslash] \end{aligned}$$

where $(\alpha = x + pw)$, $(\beta = y + ph)$.

3. ***ReLU Activation***: Discards negative feature scores:

$$\begin{aligned} & \backslash[\\ & R_{\{o\}}^{x,y,z} = \max(0, P_{\{o\}}^{x,y,z}) \\ & \backslash] \end{aligned}$$

$(R_{\{o\}}^{x,y,z})$ is used as $(i_{\{l+1\}})$ in the next CONV layer.

4. ***FC Layer Operation***: Each element of (o_i) is a weighted sum of all (i_i) elements and their biases:

$$M_{o_i}^k = \sum_{n=1}^N (i_n^i \times W_{n,k}^i) + b_k^i$$

Softmax activation is applied to compute class probabilities:

$$p_n = \frac{e^{M_{o_i}^n}}{\sum_{k=1}^N e^{M_{o_i}^k}}$$

The CNN accelerator returns the prediction result to the onboard computer for display.

C. Research Challenges

1. ***Energy-Efficient Data Flow***:

- ***Complexity and Efficiency***: The operations (1)-(5) vary in computational complexity. Efficient hardware implementation requires high-speed processing and energy efficiency, which can be achieved by mapping operations over a large PE array, exploiting parallelism and local data reusability.

- ***Row-Stationary Approach***: While this method reuses weights, inputs, and partial sums within the PE array, it necessitates extensive data shifting, making FPGA implementation infeasible. An alternative, the kernel partition technique, reduces memory access but introduces complex preprocessing and postprocessing.

- ***Precision Optimization***: Properly optimizing data precision across layers can store all CNN model weights in FPGA BRAM, avoiding off-chip DRAM access and enhancing energy efficiency. However, this approach requires low-level hardware redesign and model retraining to preserve accuracy.

- ***Proposed Solution***: This work presents a RALB-based modified row-stationary data flow that maximizes i - and wt reuse without shifting them between PEs, reducing latency and conserving energy.

2. ***Throughput Alleviation Due to Interruption***:

- ***Phased Computation***: Direct mapping of operations requires an enormous PE array, which is impractical due to design constraints. Instead, computations are phased over multiple iterations, producing partial sums (Psum) that are combined to generate final outputs.

- ***Idle Time and Latency***: Conventional data flow methods cause frequent PE array idling between iterations, increasing latency and degrading throughput. Approaches like the pin-pong method reduce latency but do not support local data reuse, leading to higher memory bandwidth requirements and incomplete idle time minimization.

- *Proposed Technique*: The new technique and hardware architecture optimize data fetching and storage, mitigating frequent PE array throttling and enhancing throughput and energy efficiency.

Summary

The article discusses the challenges and solutions in designing an energy-efficient, high-throughput FPGA-based CNN accelerator for object recognition. It highlights the importance of maximizing data reusability and minimizing data movement to enhance energy efficiency. The proposed RALB-based hardware architecture addresses these issues by optimizing data flow and storage, reducing latency, and improving throughput. The FPGA implementation supports reconfigurability, making it suitable for various contemporary applications while maintaining energy efficiency.

Detailed Summary

Proposed Uninterrupted-Processing Technique and VLSI Architectures

The proposed work focuses on enhancing the performance and efficiency of a CNN accelerator implemented on FPGA by introducing an uninterrupted-processing technique and a novel VLSI architecture. This section details these innovations, highlighting how they address existing challenges and improve system throughput and energy efficiency.

A. Proposed Uninterrupted-Processing Technique

Conventional vs. Proposed Approach:

- In conventional CNN accelerators, data fetching (F_i) and processing (P_i) occur sequentially. The PE array remains idle during the data fetching time (t_{fi}), leading to increased iteration time ($t_{itri} = t_{fi} + t_{ci}$).
- The proposed technique allows data fetching and processing to occur concurrently, effectively reducing the iteration time to approximately the computation time ($t_{itri} \approx t_{ci}$).

Algorithm 1: Uninterrupted-Processing Technique:

1. *Initialization*: Fetch initial data required for computation.
2. *Concurrent Execution*: Begin data computation once a minimal set of data (r) is fetched, while continuing to fetch remaining data.
3. *Prefetching*: Prefetch data for the next iteration before the current computation ends, enabling immediate transition to the next iteration without idle time.

Timing Diagram Illustration:

- *Conventional Timing:* Shows sequential F_i and P_i , resulting in idle periods.
- *Proposed Timing:* Demonstrates overlapping F_i and P_i , reducing latency and improving throughput.

Mathematical Impact:

- The latency reduction from $\lambda_c = \sum(t_{fi} + t_{ci})$ to $\lambda_p = \sum t_{ci} + t_{pf1}$, where t_{pf1} is the prefetch duration for the first iteration, significantly conserves processing time.

B. Proposed VLSI Architectures

1. Low-Latency CNN-Accelerator Architecture:

Architecture Overview:

- The CNN accelerator features an array of PEs (Processing Elements) connected to RALBs (Row-Aligned Local Buffers) for efficient data storage and reuse.
- *RALB:* Introduces dual-port memory for filter weights and a specialized approach for storing and reusing input feature maps (i -) locally within the PE array without shifting data between PEs.

PE Array Design:

- *RALB Microarchitecture:* Each RALB connects to one or more PE rows, storing i - values and enabling simultaneous read-write operations.
- *PE Microarchitecture:* Includes memory for filter weights, a computation unit for MAC and max operations, and a bus-selection control for flexible data routing.

Operational Flow:

- During an iteration, PEs read data from RALBs while RALBs continue to fetch new data, allowing uninterrupted computation.
- The RALB addresses are managed by a WA (Write Address) generator, ensuring proper data flow and avoiding read-before-write conflicts.

Example (3×3 PE Array):

- Illustrates how RALBs and PEs interact over multiple iterations, showing data storage, reuse, and address management to achieve efficient computation.

2. Data Reusability in PE Array Architecture:

Horizontal and Vertical Data Reuse:

- *Horizontal Reuse:* i- values are reused across horizontal strides without moving data between PEs.
- *Vertical Reuse:* RALBs shift connections with PE rows to reuse i- values during vertical strides, optimizing data utilization.

PE Connection Dynamics:

- The RALBs periodically reset their read addresses to manage vertical strides and ensure continuous data supply to PEs.
- This cyclical connection pattern ensures maximal reuse of i- values and filter weights, reducing memory access and enhancing energy efficiency.

Example Scenario:

- Describes the connection and data flow between RALBs and PE rows in a 3×3 array over four iterations, illustrating efficient data reuse and reduced latency.

Conclusion

The proposed uninterrupted-processing technique and VLSI architectures significantly enhance the performance of CNN accelerators on FPGA platforms. By enabling concurrent data fetching and processing, utilizing dual-port memories, and introducing efficient data reuse mechanisms, the proposed design achieves lower latency, higher throughput, and improved energy efficiency. These advancements address the limitations of conventional designs, making the proposed architecture a robust solution for high-performance object detection applications.

Summary of the Section on FPGA Implementation Results, Comparisons, and Hardware Validation

A. FPGA Implementation Results

The proposed architecture of the $m \times n$ Processing Element (PE) array in the CNN accelerator impacts three main factors:

1. Length of the vertical routing network.
2. Size of the bus selection multiplexer within the PE.
3. Data movement latency between a single Reconfigurable Addressable Logic Block (RALB) and the PE in the farthest row.

The computation throughput (T) is proportional to the number of PEs (NPE) and PE efficiency (V). The efficiency (V) is the ratio of utilized PEs (NPE_{util}) to total PEs (NPE). To maximize throughput, the PE array dimensions (m and n) must match the dominant filter shapes in CNN models. Analysis of popular CNN models shows that smaller filters (e.g., 3×3) dominate computations. Consequently, the architecture focuses on these smaller filters to enhance efficiency.

The proposed accelerator has a 36×24 PE array divided into 16 smaller clusters, each with a 9×6 PE array and nine RALBs. Each cluster can efficiently handle six 3×3 filters (100% efficiency), two 5×5 filters (92% efficiency), 54 1×1 filters (100% efficiency), or one 7×7 filter (91% efficiency). Implemented on the Xilinx Zynq-UltraScale+ MPSoC-ZCU102 board, the design operates at a maximum clock frequency of 340 MHz. Critical path analysis shows the longest delay involves two multiplexers, one adder, and one multiplier.

B. Comparison With State-of-the-Art Implementations

For fair comparison, the proposed accelerator was also implemented on other FPGA platforms (Xilinx Virtex-7 and ZYNQ-7000). On these platforms, the maximum clock frequency was 200 MHz due to slower BRAMs. Throughput (T) is calculated as $T = (2 \cdot NPE \cdot f_{clk} \cdot U \cdot V)$ GOPs, where U (activity rate of PEs) equals 1 due to nearly uninterrupted processing.

Energy efficiency (E) is defined as $E = T / W$, where W is average power consumption. The architecture minimizes costly data movements by maximizing local data reuse, leading to significant power savings. The design consumes 4.109 W on the Zynq-UltraScale+ FPGA board, with 82% dynamic power and 18% static power. This power consumption is 2.34 times lower than state-of-the-art implementations, resulting in an energy efficiency of 140.95 GOPs/W, 6.24 times higher than contemporary designs.

The accelerator was tested with VGG-16 and GoogLeNet neural networks, achieving high PE utilization and throughputs of 576.7 GOPs (202 frames/s) and 587.52 GOPs (18.9 frames/s), respectively. The figure-of-merit (FOM) η_{PE} , indicating operations per second per PE, shows the proposed design outperforms previous implementations. For example, on the Xilinx Virtex-7 VC709 board, the proposed design achieves a throughput of 345.6 GOPs and a PE efficiency of 400 MOPs/PE, surpassing the best reported results.

C. Peak Throughput Issues Due to DRAM Bandwidth Limitation of FPGA

Many CNN accelerators face challenges in achieving peak throughput due to limited DRAM bandwidth on FPGA boards. For instance, some designs need more than 64-byte parallel data transfer to reach peak throughput, and others can't fully hide external communication time under computation due to bandwidth constraints. The proposed accelerator mitigates this by extensive local data reuse, requiring a maximum of 16-byte parallel data transfer to/from DRAM, which is well within the sustainable bandwidth of the used FPGA board.

Overall, the proposed CNN accelerator architecture demonstrates significant improvements in efficiency, throughput, and power consumption compared to state-of-the-art designs, particularly benefiting from optimized local data reuse and efficient handling of dominant small filter sizes in popular CNN models.

D. Compatibility With State-of-the-Art CNN Models

The architecture supports various filter sizes (1×1 , 3×3 , 5×5 , 7×7) and can be configured for models like MobileNet-V1, MobileNet-V2, and EfficientNet. Clusters can be set for depth-wise separable convolutions and point-wise convolutions, working in a pipelined manner to minimize off-chip data access. It also supports residual connections with slight modifications.

E. Hardware Validation of the Proposed CNN Accelerator

Hardware Test Setup and Validation Method:

The setup includes a host computer, a Zynq-UltraScale+ MPSoC-ZCU102 FPGA board, and an external SD memory card.

Model parameters and input images are stored on the SD card, converted to FP16 format.

The FPGA board processes data using the CNN accelerator, interfacing with onboard DRAM via DMA and AXI ports.

A C-language application on the host computer handles data transfers and configurations.

Experimental Results:

The proposed architecture was tested with GoogLeNet on an RGB image of dimension $224\times224\times3$.

Layer-wise processing includes convolution operations with filters of various sizes, ReLU activations, and max pooling, with intermediate results visualized as images.

The output of the final fully connected layer undergoes softmax activation to classify the input image, validating correct functionality.

Accuracy:

The design supports both BF16 and FP16 precision. FP16 was chosen for easier FPGA implementation, though it results in a slight accuracy reduction (0.5%).

Summary

****A Dual-Stream Convolution-GRU-Attention
Network for Automatic Modulation Classification****

1. Introduction

Automatic Modulation Classification (AMC) is crucial in wireless communications, especially for technologies like cognitive radio and adaptive modulation. AMC enables the receiver to identify the modulation scheme of incoming signals, essential for both military and commercial applications. This task is challenging due to the absence of prior knowledge about transmitted data and unknown parameters such as phase offsets, signal power, timing, and carrier frequency. Traditional AMC methods, including likelihood-based and fuzzy methods, have limitations, particularly in low Signal-to-Noise Ratio (SNR) conditions. Recently, deep learning (DL) techniques have been explored due to their superior performance over traditional methods. This paper proposes a novel hybrid dual-stream architecture combining Convolutional Neural Network (CNN) and Gated Recurrent Unit (GRU) layers, enhanced with an attention mechanism, to classify seven digital modulation types.

2. Signal Model

The received signal model is given by:

$$r_x(t) = \exp[j \cdot 2\pi f(t)] \int_0^{\tau_0} s[2\pi c(t - \tau)] h(t, \tau) d\tau + n(t)$$

where j is an imaginary number, $2\pi f(t)$ is the carrier frequency offset, $2\pi c(t)$ is the sampling rate offset, τ_0 is the maximum delay spread, $s(t)$ is the transmitted signal, $h(t, \tau)$ is the fading channel, and $n(t)$ is the additive white Gaussian noise (AWGN). The discrete form of the received signal is used, containing in-phase (r_I) and quadrature (r_Q) components. The dataset used is RML2016.10B, focusing on digital modulations: QAM16, CPFSK, 8PSK, QAM64, GFSK, BPSK, and QPSK.

3. Signal Preprocessing

Two data representations are used: In-phase/Quadrature (I/Q) and Amplitude/Phase (A/P).

1. *I/Q Representation*:

$$\mathbf{X}_{I/Q} = \begin{bmatrix} X_I \\ X_Q \end{bmatrix}$$

where X_I and X_Q are the in-phase and quadrature components.

2. *A/P Representation*:

$$\mathbf{X}_{A/P} = \begin{bmatrix} X_A \\ X_P \end{bmatrix}$$

where X_A (amplitude) and X_P (phase) are calculated as:

$$X_A = \sqrt{r_I^2 + r_Q^2}, \quad X_P = \arctan\left(\frac{r_Q}{r_I}\right)$$

4. Proposed Method

The architecture consists of two parallel streams, each with similar components:

1. *Convolutional Layer*:

- 256 filters of size 2×3
- ReLU activation
- Dropout layer (rate 0.5)
- Zero padding for consistent output shape

2. *GRU Layer*:

- 50 memory units
- Dropout rate 0.5

3. *Attention Layer*:

- Processes sequences of data for better feature extraction

The output features from both streams are combined using an outer product operation to enhance feature diversity, followed by flattening and classification using a dense softmax layer.

5. Results and Discussions

The dataset is divided into 80% for training and 20% for testing. Training parameters include a batch size of 1024, 100 epochs, and the Adam optimizer with a learning rate of 0.001. The training and validation accuracy indicate no overfitting, with accuracy improving up to around

80 epochs. The model achieves over 75% accuracy across the SNR range from -4 dB to 18 dB and surpasses 96% accuracy from 2 dB to 18 dB SNR.

Comparative analysis shows the proposed model outperforms other models like CNN, CNN2, CLDNN, and CNN-LSTM in terms of classification accuracy across a wide SNR range. The confusion matrix indicates high accuracy, even in low SNR conditions. The proposed model also demonstrates lower complexity, faster training, and testing times compared to other models.

6. Conclusion

The hybrid dual-stream CNN and GRU model with an attention mechanism effectively captures spatial and temporal features, improving classification accuracy for digital modulation schemes across various SNRs. The proposed model's efficiency and reduced complexity make it suitable for real-time applications, outperforming state-of-the-art models in terms of accuracy and computational efficiency.

Summary of "Automatic Modulation Recognition: An FPGA Implementation" by Satish Kumar, Rajarshi Mahapatra, and Anurag Singh

Abstract

The paper addresses the challenge of deploying CNN-based automatic modulation classification (AMC) systems on FPGAs due to the large model size and floating-point operations. The authors propose a solution involving quantized CNNs and low-precision mathematics to reduce the hardware resource requirements while maintaining classification accuracy. They introduce an iterative pruning-based training mechanism to further optimize the models. The proposed scheme achieves a significant reduction in hardware resources and maintains high accuracy, achieving real-time throughput on an RFSoc platform.

Introduction

Automatic modulation classification (AMC) is critical for identifying modulation schemes without prior knowledge of various signal parameters. Traditional methods like statistical ML approaches and maximum likelihood-based strategies face limitations due to the requirement for prior knowledge and sensitivity to various errors. Recently, deep learning (DL) models, particularly convolutional neural networks (CNNs), have shown promise in AMC tasks. However, deploying these models on resource-constrained hardware like FPGAs is challenging due to their large size and computational demands. This work proposes optimized CNN-based AMC schemes for FPGA implementation, focusing on model compression, quantization, and iterative pruning to balance resource constraints and classification accuracy.

System Model

Neural Network on FPGAs

Implementing CNNs on FPGAs requires reducing the computational and memory overhead. This can be achieved through model compression and quantization. The paper employs quantization-aware training (QAT) using Brevitas with PyTorch to reduce precision and prune the models without significantly impacting accuracy.

Convolutional Layer Based Network (QMCNet)

QMCNet is a proposed AMC model designed for FPGA implementation, featuring a compact architecture with six 1D convolutional layers using smaller kernel sizes and fewer output channels. Quantization of inputs, weights, and activations further reduces the memory footprint. Despite a minor accuracy degradation, QMCNet offers significant reductions in model size and computational complexity.

Residual Unit Based Network (RUNet)

To address the accuracy degradation seen in QMCNet, the authors propose RUNet, which employs deeper networks and residual units (RU) to maintain accuracy while optimizing complexity. Residual units use skip connections to enhance feature extraction and prevent overfitting. The RUNet model applies 6-bit quantization to inputs, weights, and activations.

Training and Pruning Methodology

Iterative pruning is used during training to reduce the number of trainable parameters and computational complexity. The method involves removing neurons with negligible weights and redistributing their contributions to other weights. This process iteratively prunes and retrains the network to maintain accuracy above a certain threshold (58% in this case).

Simulation Results

Classification Performance

The proposed AMC schemes are evaluated using the RadioML 2018.01A dataset. The QMCNet and RUNet models show competitive accuracy across a range of SNRs, with RUNet achieving higher accuracy than the baseline and QMCNet models. The confusion matrices highlight the classification performance for various modulation schemes, showing improved accuracy for higher-order QAM modulations in the RUNet model.

Model Complexity

The complexity analysis considers the number of multiply-accumulate (MAC) operations, parameter counts, and bit-operations. The proposed models significantly reduce these metrics compared to the baseline, with QMCNet and RUNet models achieving substantial reductions in bit-operations and memory requirements.

Hardware Implementation

Resource Utilization

The implementation on the RFSoc ZCU111 platform shows that the proposed QMCNet and RUNet models efficiently utilize hardware resources. The resource consumption metrics indicate that the proposed models require significantly fewer CLBs and LUTs compared to the baseline model.

Conclusion

The paper demonstrates the feasibility of real-time implementation of DL-based AMC models on RFSoc platforms. By employing quantized CNNs, QAT, and iterative pruning, the authors achieve substantial reductions in model complexity and memory requirements while maintaining high classification accuracy. The proposed models, particularly RUNet, deliver competitive accuracy with high throughput and low latency, making them suitable for deployment in resource-constrained environments.

Key Contributions

- *Quantized CNNs and Low-Precision Mathematics*: The authors propose using quantized CNNs with low-precision weights and activations to reduce hardware resource requirements.
- *Iterative Pruning-Based Training*: A pruning methodology is used to further optimize the models by eliminating negligible weights and retraining the network to maintain accuracy.
- *Efficient FPGA Implementation*: The proposed models are implemented on the RFSoc ZCU111 platform, demonstrating significant reductions in resource utilization while maintaining high classification accuracy and throughput.

Significance

This work provides a practical approach to deploying DL-based AMC systems on FPGAs, addressing the challenges of model size and computational demands. The proposed methodologies enable efficient utilization of hardware resources, making it feasible to implement high-performing AMC systems in real-time on resource-constrained platforms.

Modulation Classification for Non-Orthogonal Multiple Access System using a Modified Residual-CNN

Ashok Parmar¹, Kamal Captain¹, Udit Satija², and Ankit Chouhan¹

¹Department of Electronics Engineering, Sardar Vallabhbhai National Institute of Technology, Surat, India

{ds20ec010, kamalcaptain, d20ec003}@eced.svnit.ac.in

²Department of Electrical Engineering, Indian Institute of Technology Patna, Bihar, India
udit@iitp.ac.in

Abstract

Non-orthogonal multiple access (NOMA) is a promising solution to the problem of spectrum deficiency. NOMA receivers require information about the modulation type of the co-scheduled user's signal to perform successive interference cancellation (SIC). Automatic modulation classification (AMC) is used to reduce the signal overhead created due to sharing of this information. In this paper, a convolutional neural network (CNN) with a modified residual block (MR-CNN) is proposed for AMC in NOMA systems. The classification performance of MR-CNN is evaluated on the input signal with four different modulation formats at varying signal to noise ratios (SNRs). Experimental results demonstrate that the proposed model can achieve more than 90% classification accuracy for SNRs higher than 10dB. Further, comparative analysis depicts that the proposed model outperforms existing methods in terms of classification accuracy at low SNRs and has similar performance at high SNRs.

Index Terms

Non-orthogonal multiple access (NOMA), automatic modulation classification (AMC), convolutional neural network (CNN), convolutional residual block.

I. Introduction

The next-generation wireless network demands low latency, improved fairness, high reliability, massive connectivity, and high spectrum efficiency. Non-orthogonal multiple access (NOMA) is an emerging technology that helps to meet these demands. In NOMA, multiple users' signals are superposed in time, frequency, and power domains, significantly boosting spectrum efficiency. Signals are superposed with power differences based on the channel conditions of receivers. Higher power is allocated to the user with lower channel gain (i.e., the far user). To recover the signal, the far user treats the near user's signal as noise, while the near user needs to apply successive interference cancellation (SIC) to cancel the far user's signal and recover its own signal. This requires the modulation information of the far user's signal, which can create extra overhead if sent via signaling. Automatic modulation classification (AMC) can reduce this overhead.

Existing AMC methods primarily focus on single-user scenarios in orthogonal multiple access (OMA) systems without co-channel interference. These methods are categorized into likelihood-based (LB) and feature-based (FB) methods. While LB methods use probabilistic approaches, FB methods employ feature extraction. LB methods perform well for fewer modulations and high SNRs but degrade with more modulations and are computationally complex. FB methods, with lower computational complexity, have been improved using various techniques like higher-order cumulants, machine learning classifiers, and deep learning (DL).

DL methods have shown promising results in OMA but are less explored for NOMA. This work proposes a CNN architecture with a modified residual block (MR-CNN) to classify the modulation type of the far user's signal at the near user's end. The CB method is used as a benchmark for comparative analysis.

II. System Model

The NOMA scheme in downlink with superposed signals of two users is considered, where the received signal at time instant t is given by:

$$y(t) = h \sqrt{P_f} x_f(t) + \sqrt{P_n} x_n(t) + w(t)$$

Here, $x_f(t)$ and $x_n(t)$ are unit energy signals for the far and near user, respectively. h represents the Rayleigh fading channel gain between the base station (BS) and the near user terminal, and $w(t)$ is the additive white Gaussian noise (AWGN) with zero mean and variance σ^2 . P_f and P_n are the power allocated to the far and near user, respectively, with $P_f + P_n = 1$. The near user applies SIC to cancel the far user's signal using the modulation type information for CWIC.

III. Proposed Method

The proposed MR-CNN architecture for AMC includes four convolution layers, two average pooling layers, three dense layers, and two modified residual blocks. Each convolution and dense layer uses ReLU activation, except the output dense layer, which uses softmax. The input is reshaped to $(2 \times N \times 1)$ before feeding to the first convolution layer.

The modified residual block has two parallel convolution layers: one with filters of size (1×8) in the main path and another with (1×1) size filters in the skip path. The skip connection with ReLU activation helps avoid vanishing gradient problems and allows learning non-linear features.

IV. Results and Discussion

A. Performance Evaluation

The dataset, generated synthetically, includes signals with varying SNRs. The classification accuracy improves with increasing SNR and power ratio. For $(N = 200)$, the accuracy exceeds 95% for SNRs higher than 8dB. Increasing the number of samples (N) improves accuracy but also increases computational complexity.

B. Comparative Analysis with CB Method

The MR-CNN shows better classification accuracy at low SNRs compared to the CB method and performs similarly at high SNRs. The confusion matrices indicate the probabilities of correctly predicting the modulation type, with BPSK achieving high accuracy even at low SNRs.

V. Conclusion

This work introduces a CNN architecture with a modified residual block for AMC in NOMA systems. The proposed MR-CNN achieves high classification accuracy at varying SNRs, outperforming existing methods at low SNRs. Future work will focus on extending the model to handle more users and modulation schemes.

Summary

Introduction

Automatic Modulation Classification (AMC) is essential for various applications such as monitoring spectrum interference, dynamic spectrum access, and military uses. Traditional approaches to AMC involve handcrafted feature extraction and decision boundaries, but recent advancements in deep learning (DL) show promise for significant performance improvements. This paper explores the application of Convolutional Neural Networks (CNNs) in DL for AMC, proposing a novel architecture that outperforms existing models.

Related Work

Several CNN-based architectures for AMC have been proposed:

1. **VTCNN2**: Lacks pooling and regularization layers, which are essential for reducing computational costs and overfitting.
2. **Various ANN Models**: Require significant preprocessing for feature extraction.
3. **Other CNN Architectures**: Include convolution layers and fully connected layers with dropout for regularization, but could benefit from noise layers for robustness.

The research indicates that CNNs, especially when combined with long-term memory layers like LSTM, can significantly improve AMC performance.

Problem Statement

AMC is a classification problem where the goal is to identify the modulation type of a received signal, $y(t)$, based on its observed values. The challenge is to accurately classify the signal type in the presence of noise and channel effects.

Dataset Description

The RadioML 2016.04C dataset is used for training and evaluating the model. It includes:

- **Modulation Types**: 11 different types, including AM-DSB, AM-SSB, BPSK, etc.
- **Sample Length**: 128

- *SNR Range*: -20 dB to +18 dB
- *Training Samples*: 90,753
- *Validation Samples*: 38,895
- *Test Samples*: 32,412

The dataset includes various signal-to-noise ratios (SNR) and channel imperfections, making it suitable for robust model training.

Proposed Architecture

The proposed CNN architecture consists of six stages:

1. *Stage 1*: Zero-padding followed by a convolution layer with 256 filters and a kernel size of (1,2), ReLU activation, and dropout.
2. *Stage 2*: Average pooling layer with a kernel size of (1,2) for reducing parameters and improving robustness.
3. *Stage 3*: Zero-padding followed by a convolution layer with 96 filters and a kernel size of (1,4), ReLU activation, and dropout.
4. *Stage 4*: Zero-padding followed by a convolution layer with 64 filters and a kernel size of (2,6), ReLU activation, and dropout.
5. *Stage 5*: Flattening layer followed by a Gaussian noise layer to improve robustness and regularization.
6. *Stage 6*: Fully connected layers with 1024, 256, 64, and 11 nodes, followed by a SoftMax activation layer for classification.

Results

Training and Validation

- *Training*: The model achieved a training accuracy of 70.60% and a loss of 0.7832.
- *Validation*: The best validation accuracy was 69.10% with a loss of 0.8206.

Testing

- ***Test Accuracy***: The proposed model achieved 68.946% test accuracy with a loss of 0.8242.

- ***Confusion Matrix***: At high SNR values, the model showed nearly perfect classification, but some confusion remained between certain modulation types (e.g., WBFM and AM-DSB).

Comparison

The proposed model outperformed several existing models, achieving a test accuracy of 68.94% and a validation loss of 0.8242. At 18 dB SNR, the accuracy was 97.24%, higher than other models such as VTCNN2 and other CNN-based architectures.

Conclusion

The proposed CNN-based architecture for AMC demonstrates high classification accuracy and effective regularization using Gaussian noise layers. The results indicate that the proposed model not only surpasses existing state-of-the-art CNN architectures but also provides a more robust and efficient solution for automatic modulation classification.

A Dual-Stream Convolution-GRU-Attention Network for Automatic Modulation Classification:

The hybrid dual-stream CNN and GRU model with an attention mechanism effectively captures spatial and temporal features, improving classification accuracy for digital modulation schemes across various SNRs. The proposed model's efficiency and reduced complexity make it suitable for real-time applications, outperforming state-of-the-art models in terms of accuracy and computational efficiency.

Automatic Modulation Recognition: An FPGA Implementation:

The paper demonstrates the feasibility of real-time implementation of DL-based AMC models on RFSoc platforms. By employing quantized CNNs, QAT, and iterative pruning, the authors achieve substantial reductions in model complexity and memory requirements while maintaining high classification accuracy. The proposed models, particularly RUNet, deliver competitive accuracy with high throughput and low latency, making them suitable for deployment in resource-constrained environments.

Key Contributions

- **Quantized CNNs and Low-Precision Mathematics:** The authors propose using quantized CNNs with low-precision weights and activations to reduce hardware resource requirements.
- **Iterative Pruning-Based Training:** A pruning methodology is used to further optimize the models by eliminating negligible weights and retraining the network to maintain accuracy.
- **Efficient FPGA Implementation:** The proposed models are implemented on the RFSoc ZCU111 platform, demonstrating significant reductions in resource utilization while maintaining high classification accuracy and throughput.

Significance

This work provides a practical approach to deploying DL-based AMC systems on FPGAs, addressing the challenges of model size and computational demands. The proposed methodologies enable efficient utilization of hardware resources, making it feasible to implement high-performing AMC systems in real-time on resource-constrained platforms.

Modulation Classification for Non-orthogonal Multiple Access System using a Modified Residual-CNN:

This work introduces a CNN architecture with a modified residual block for AMC in NOMA systems. The proposed MR-CNN achieves high classification accuracy at varying SNRs, outperforming existing methods at low SNRs. Future work will focus on extending the model to handle more users and modulation schemes.

Automatic Modulation Classification: A Novel Convolutional Neural Network Based Approach:

The proposed CNN-based architecture for AMC demonstrates high classification accuracy and effective regularization using Gaussian noise layers. The results indicate that the proposed model not only surpasses existing state-of-the-art CNN architectures but also provides a more robust and efficient solution for automatic modulation classification.

An Uninterrupted Processing Technique-Based High-Throughput and Energy-Efficient Hardware Accelerator for Convolutional Neural Networks: